UI-Design für Ingenieuranwendungen

Joern Ploennigs · AI4SC



> Design is not just what it looks like and feels like. Design is how it works.
>
> — Steve Jobs

500 Standorte, ein CSV — was jetzt?

Das Ergebnis liegt vor:

- 500 Zeilen · 12 Spalten · keine Legende
- Kritische Grenzwertüberschreitungen — irgendwo darin
- Nächste Entscheidung: Welche 20 Standorte priorisieren?

Ohne Interface bleibt die Frage offen.

Eine Zahl ohne Einheit ist keine Messung.

Ein Messwert ohne Kontext ist keine Information.

Eine Information ohne Interface ist keine Entscheidungsgrundlage.

Ein typischer Fehler im Ingenieurstudium: die Berechnung als Endziel zu betrachten. In der Praxis ist die Berechnung ein Zwischenschritt. Das Ergebnis muss für die Zielgruppe — Behörden, Fachkollegen, Management — aufbereitet werden. Die Wahl des Ausgabeformats ist eine Ingenieurentscheidung, keine nachgelagerte Nebensache.

Cross-Referenz: Die Rasterberechnungen aus `11a_Raster_Simulation` sind ein typisches Beispiel — ein numpy-Array ist kein Ergebnis für einen Stadtrat.

UI-Typen

TL;DR
Verschiedene Ausgabeformate bedienen verschiedene Zielgruppen und Anwendungsfälle. Streamlit und Gradio ermöglichen interaktive Web-UIs direkt aus Python — ohne Webentwicklungskenntnisse.

UI-Typen im Vergleich

Format	Interaktiv	Zielgruppe	Typischer Einsatz
Konsole / Terminal	—	Entwickler	Tests, Skripts
Jupyter Notebook	begrenzt	Ingenieure	Analyse, Dokumentation
Streamlit Web-App	**ja**	**Fachpublikum**	**Dashboards, Prototypen**
Gradio Interface	**ja**	**Endnutzer**	**ML- und Modell-Schnittstellen**
PDF-Report	—	Management	Offizielle Berichte

Jedes Format hat seinen Platz im Ingenieuralltag. Für schnelle Exploration intern eignet sich Jupyter. Wenn ein Ergebnis mit einer Fachabteilung oder einem Auftraggeber geteilt werden soll, erlauben Streamlit oder Gradio einen professionelleren Eindruck bei minimalem Zusatzaufwand. Für rechtlich relevante Dokumentation ist der PDF-Report meist obligatorisch.

Die Entscheidung für ein Format ist keine rein technische: Sie hängt davon ab, wer das Ergebnis wann und in welchem Kontext sehen soll — ein Kernaspekt des Design-Thinking-Prozesses.

Design-Prinzipien

TL;DR
Gutes Interface-Design folgt messbaren Prinzipien: Klarheit, Präzision, konsistente Bedeutung. Design Thinking strukturiert den Designprozess nutzerorientiert — und verhindert, dass Ingenieure Dashboards bauen, die nur sie selbst verstehen.

Design Thinking für Ingenieur-UIs

Prozess: Empathize → Define → Ideate → Prototype → Test
Design Thinking ist ein strukturierter Ansatz zur nutzerorientierten Gestaltung von Systemen, der ursprünglich aus dem Produktdesign stammt. Für Ingenieur-UIs ist er besonders wertvoll, weil technisch korrekte Ausgaben oft für die falsche Zielgruppe optimiert sind.

Empathize: Führe ein kurzes Interview mit der Zielgruppe durch. Welche Fragen stellt sie täglich? In welchem Kontext öffnet sie das Dashboard — am Desktop, im Außeneinsatz, im Meeting?

Define: Formuliere eine klare Problemaussage: „Der Standortleiter braucht auf einen Blick zu sehen, welche Standorte heute den Grenzwert für PM2.5 überschreiten." Das begrenzt den Scope.

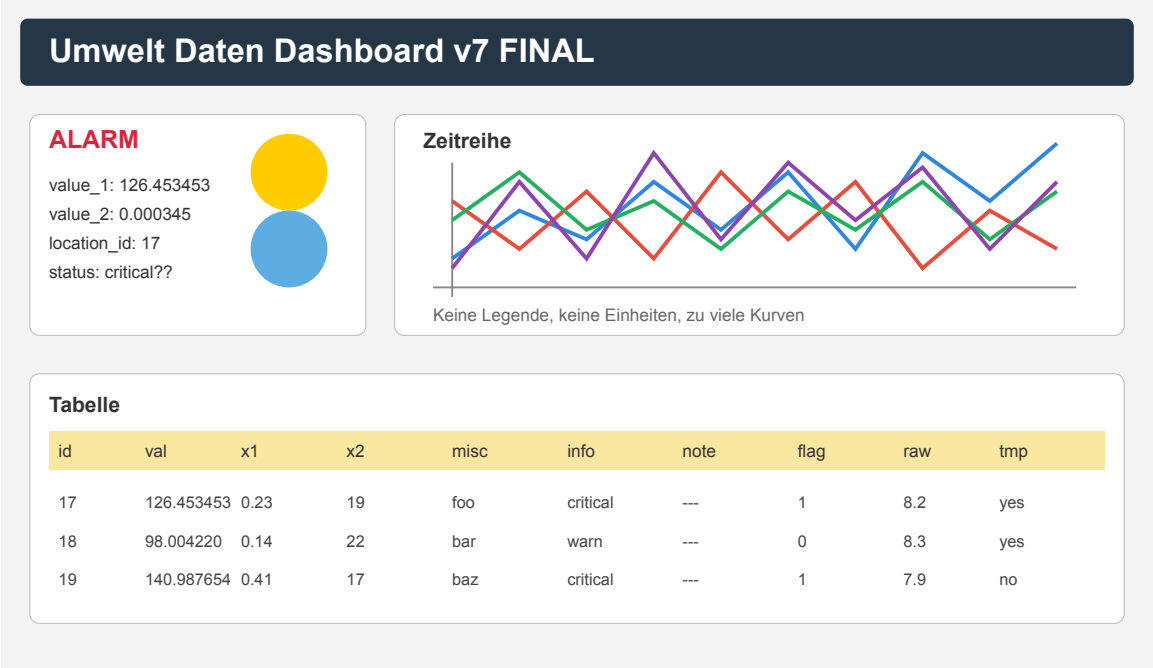
Ideate: Erst auf Papier skizzieren — Wireframe, kein Code. Schlechte Ideen findet man mit einem Bleistift schneller als nach drei Stunden Streamlit-Coding.

Prototype: Streamlit und Gradio sind ideal für schnelle Prototypen. Ein KI-generiertes Grundgerüst kann in 30 Minuten für einen ersten Nutzerfeedback-Loop bereit sein.

Test: Beobachte, wie die Zielgruppe das Interface nutzt. Stellt sie Fragen, die das Interface beantworten sollte? Dann fehlt Information. Ignoriert sie Elemente? Dann sind es zu viele.

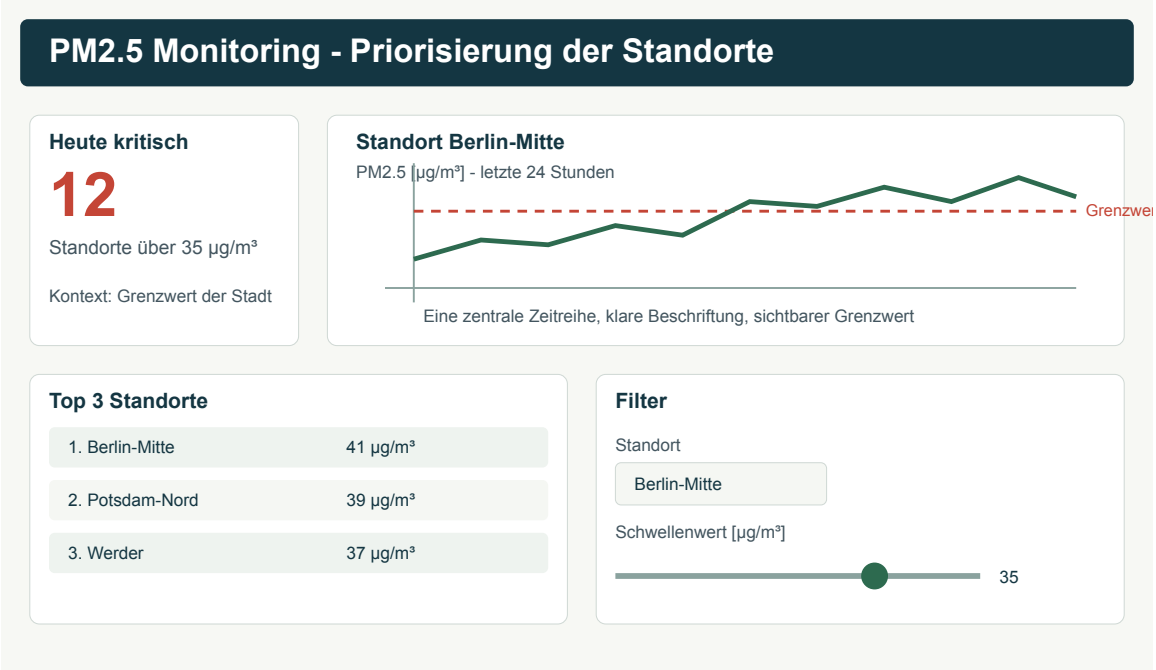
Gute und schlechte Dashboards vergleichen

Schlechtes Dashboard



- kryptische Labels wie `value_1`
- keine Einheiten, keine Legende
- zu viele gleichzeitige Signale

Besseres Dashboard

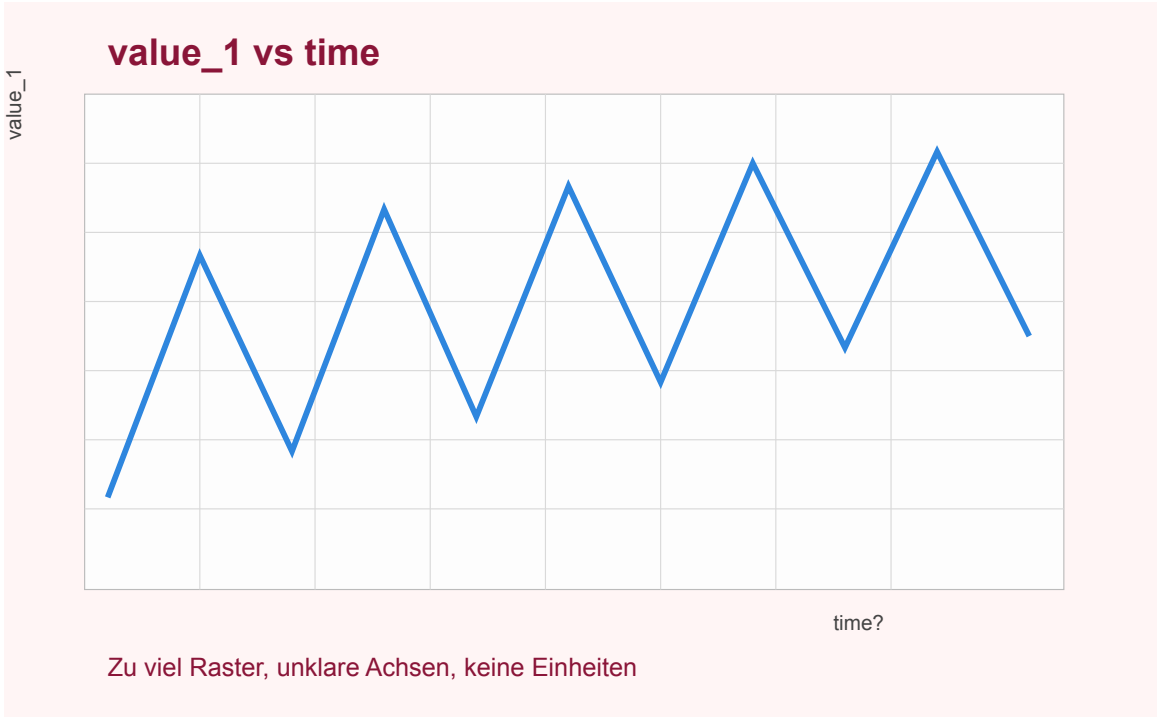


- klare Priorisierung der Kernfrage
- sichtbarer Grenzwert und Einheiten
- wenige, semantisch geordnete Elemente

Der Vergleich schult den Blick: Beide Dashboards könnten dieselben Daten enthalten, aber nur das rechte unterstützt eine belastbare Entscheidung. Das schlechte Beispiel verletzt mehrere Grundsätze gleichzeitig: fehlende Einheiten, unklare Benennung, keine Informationshierarchie und visuelle Überladung. Das bessere Beispiel beantwortet zuerst die operative Frage: *Wo muss ich handeln?*

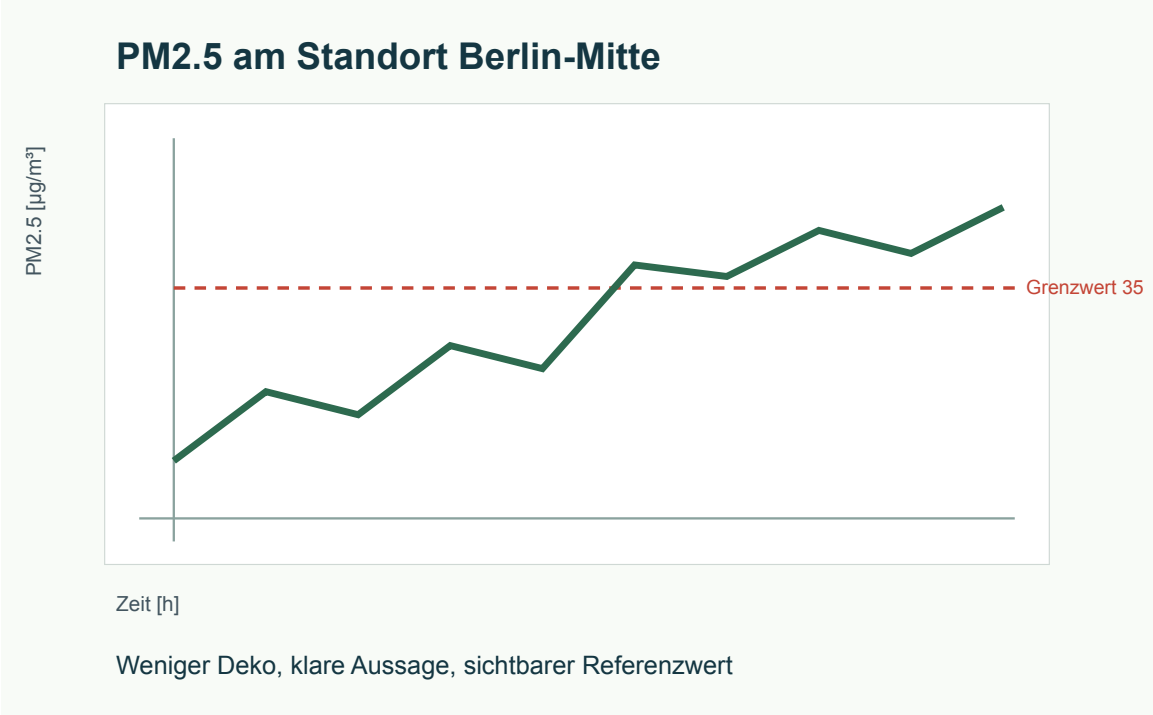
Gute und schlechte Diagramme vergleichen

Schlechtes Diagramm



- Achsen ohne fachliche Bedeutung
- viel Deko, wenig Orientierung
- Referenzwert fehlt

Besseres Diagramm



- Achsen mit Größe und Einheit
- Grenzwert sichtbar markiert
- eine erkennbare Kernaussage

Gerade bei Messdaten entscheidet oft nicht der Plot selbst, sondern der Kontext im Plot. Ein Grenzwert, eine Einheit und ein sprechender Titel verwandeln eine Kurve in eine fachlich interpretierbare Aussage.

Grundsätze I: Klarheit und Präzision

Klarheit

- Ein Diagramm = eine Aussage
- Keine 10 Kurven in einem Chart
- Wichtigstes zuerst (*Informationshierarchie*)

Präzision

- Einheiten immer: „PM2.5 [µg/m³]“, nicht „value_1“
- Signifikante Stellen: 2,34 mg/l — nicht 2,3400000
- Achsengrenzen: physikalisch sinnvoll, kein Auto-Scale

Informationshierarchie

- wichtigste Kennzahl nach oben links
- hoher Kontrast für kritische Werte
- Details erst nachrangig sichtbar machen

Affordances

- ein Slider sollte wie ein Slider aussehen
- Buttons eindeutig beschriften
- Interaktion muss erwartbar wirken

Grundsätze II: Wahrnehmung und Reduktion

Gestalt-Prinzipien

- *Proximity:* Zusammengehörige Werte beieinander
- *Similarity:* Gleiche Farbe = gleiche Bedeutung
- *Figure/Ground:* Daten vom Hintergrund abheben
- *Continuity:* Linien als zusammenhängende Verläufe nutzen

Data-Ink-Ratio *(Tufte)*

- Minimaler dekorativer Aufwand
- Maximaler Informationsgehalt
- Jedes Element muss eine Funktion erfüllen

Nielsen-Heuristiken: Anschluss an HCI-Grundlagen

Unser Fokus in dieser Vorlesung	Nielsen-Heuristik
klare Labels, Einheiten, Rückmeldung	1. Sichtbarkeit des Systemstatus
bekannte Begriffe wie „PM2.5 [µg/m³]“ statt `value_1`	2. Übereinstimmung mit der realen Welt
vertraute Widgets und klare Bedienung	4. Konsistenz und Standards
wenig Deko, klare Fehlerhinweise, sinnvolle Defaults	8. Ästhetisches und minimalistisches Design
verständliche Fehlermeldungen bei falschen CSVs	9. Fehler erkennen, diagnostizieren, beheben

Unsere Regeln sind also keine Geschmackssache, sondern lassen sich auf etablierte HCI-Heuristiken abbilden.

Ref.: Tufte, E. R.: The Visual Display of Quantitative Information, Graphics Press, 1983

Die Nielsen-Heuristiken sind ein klassischer Referenzrahmen aus der Human-Computer-Interaction. Für diese Vorlesung reichen wenige direkte Zuordnungen: Wenn du Einheiten sichtbar machst, den Systemzustand rückmeldest und sprechende Begriffe verwendest, arbeitest du nicht nur „schön“, sondern heuristisch fundiert.

Edward Tuftes Konzept der *Data-Ink-Ratio* besagt, dass jeder Tropfen Tinte auf einem Diagramm entweder Datenwert trägt oder entfernt werden sollte. Gitternetzlinien, überflüssige Rahmen, übermäßige Legenden — alles Rauschen, das die eigentliche Information überlagert.

Die Gestalt-Psychologie liefert Erklärungen, warum Menschen visuelle Muster erkennen. **Proximity:** Elemente, die räumlich beieinander liegen, werden als zusammengehörig wahrgenommen — nutze das für Gruppierungen im Dashboard. **Similarity:** Gleiche Farbe signalisiert die gleiche Kategorie. Wenn Rot auf einem Slide „Grenzwert überschritten“ bedeutet, muss Rot auf allen Slides dasselbe bedeuten. **Continuity:** Linien und Kurven werden als zusammenhängende Einheit wahrgenommen — daher sind Zeitreihendaten als Liniendiagramm intuitiver als als Balken.

Informationshierarchie bedeutet: Die wichtigste Information bekommt den prominentesten Platz, die größte Schrift, den höchsten Kontrast. Sekundäre Details sind sichtbar, aber nicht dominant.

Affordances (Handlungsaufforderungen) — ein weiteres zentrales UI-Konzept — beschreiben, ob Interaktionselemente intuitiv nutzbar sind: Ein Slider sollte wie ein Slider aussehen, ein Button wie ein Button. Streamlit und Gradio erzeugen korrekte Affordances automatisch; der Ingenieur muss nur die Beschriftung liefern.

Streamlit Demo

TL;DR
KI generiert ein Streamlit-Grundgerüst in wenigen Sekunden. Die kritische Ingenieurarbeit beginnt danach: Stimmen die Spaltennamen? Sind Einheiten im Label? Ist die Schwellenwert-Logik korrekt?

Agentischer Workflow: Streamlit-Dashboard

Prompt an die KI:

> *„Generiere eine Streamlit-App, die Sensorzeitreihendaten aus einer CSV-Datei lädt, nach Standort filtert und mit einem einstellbaren Schwellenwert als Linie im Plotly-Diagramm anzeigt.“*

Prompt → Generate → Understand → Verify → Iterate

KI liefert ein funktionierendes Grundgerüst in < 1 min.

Der Ingenieur trägt Verantwortung für das Ergebnis.

Nicht: „Läuft es?“

Sondern: „Ist das Ergebnis korrekt?“

- Labels richtig? ✓
- Einheiten angegeben? ✓
- Daten korrekt gemappt? ✓
- Schwellenwert sinnvoll? ✓

Der agentische Zyklus beim UI-Design folgt demselben Muster wie bei jeder anderen KI-unterstützten Programmieraufgabe: Erst einen klaren, vollständigen Prompt formulieren — je präziser die Anforderungen, desto besser das Grundgerüst. Dann den generierten Code verstehen (nicht

nur ausführen), und schließlich systematisch verifizieren, ob das Interface das zeigt, was es zeigen soll. Iteriere, bis das Interface für die Zielgruppe geeignet ist.

Gute Prompts für UI-Generierung benennen: Datenformat (CSV, DataFrame), relevante Spalten, gewünschte Interaktionselemente (Slider, Dropdown, Upload), und Ausgabeformat (Plotly-Chart, Tabelle, Download-Button).

Vom Prompt zum ersten Dashboard

1. Daten laden

- CSV auswählen
- Spalten prüfen
- Standort auswählen

2. Messreihe zeigen

- Zeit auf X-Achse
- Messwert auf Y-Achse
- Titel pro Standort

3. Schwellenwert prüfen

- Slider für Grenzwert
- rote Referenzlinie
- Plot gegen Fachfrage lesen

Als worked example lesen wir das Dashboard in drei Schritten: erst Daten laden, dann die Messreihe darstellen, dann einen Schwellenwert zur Entscheidung einblenden. Diese Zerlegung hilft dabei, den generierten Code nicht als monolithischen Block, sondern als Folge fachlicher UI-Bausteine zu verstehen.

KI-generierter Code: erster Prototyp

```
# KI-generiert — Streamlit Sensor-Dashboard
import streamlit as st
import pandas as pd
import plotly.express as px

st.title("Sensor-Monitoring")

file = st.file_uploader("CSV laden", type="csv")
if file:
    df = pd.read_csv(file)
    standort = st.selectbox("Standort", df["location"].unique())
    df_filtered = df[df["location"] == standort]
    threshold = st.slider("Schwellenwert", 0, 500, 100)
    fig = px.line(df_filtered, x="timestamp", y="value",
```

```
                  title=f"Messwerte – {standort}")
    fig.add_hline(y=threshold, line_color="red")
    st.plotly_chart(fig)
```

> Ingenieur prüft (Pattern C)
>
> Was passiert, wenn deine CSV die Spalte `messwert` heißt, nicht `value`?
>
> Welche Einheit hat der Schwellenwert? Steht sie im Label oder Diagramm?

Dieser Code zeigt ein typisches KI-generiertes Streamlit-Grundgerüst. Lies ihn in drei Blöcken: Datei laden, Standort filtern, Grenzwert visualisieren. Er funktioniert — unter der Annahme, dass die CSV-Datei Spalten namens `location`, `timestamp` und `value` enthält. In der Praxis heißen diese Spalten oft anders.

Pattern C — AI-Code Reading: Bevor du den Code ausführst:

1. Welche Spaltennamen erwartet der Code? (`df["location"]`, `df["timestamp"]`, `df["value"]`)
2. Was passiert, wenn eine davon fehlt? (KeyError — kein hilfreicher Fehler für den Endnutzer)
3. Wird die Einheit des Schwellenwerts dem Nutzer kommuniziert? (Nein — das Label lautet nur „Schwellenwert“)
4. Ist die Y-Achse mit Größe und Einheit beschriftet? (px.line verwendet den Spaltennamen „value“ — nicht informativ)

Diese vier Fragen sind die Kern-Verifikation vor dem Deployment. Iteriere dann mit einem präziseren Prompt: *„Benenne die Y-Achse mit ‚PM2.5 [µg/m³]‘ und zeige die Einheit des Schwellenwerts im Slider-Label an.“*

Streamlit verifizieren — was der Ingenieur prüft

Vier Verifikationsdimensionen:

1. **Datenmapping** ist die kritischste Prüfung. Ein korrekter Code mit falschen Spaltennamen zeigt Unsinn — aber korrekt formatiert. Vergleiche jeden Spaltennamen im Code mit der tatsächlichen Datenstruktur.
2. **Darstellung:** Achsenbeschriftungen mit Einheiten sind nicht optional. „value“ ist keine Information. „PM2.5 [µg/m³]“ ist eine Information.
3. **Logik:** Teste mit synthetischen Daten, bei denen du das erwartete Ergebnis kennst. Wenn der Grenzwert 50 µg/m³ beträgt und deine Testdaten Werte von 30, 60 und 80 haben, sollten genau zwei Punkte als überschritten markiert sein.
4. **Nutzerführung:** Ein Ingenieur kann einen Python-Traceback lesen. Ein Stadtrat nicht. Verifikation schließt ein, ob Fehlerzustände für die Zielgruppe verständlich kommuniziert werden.

Gradio: Interaktive Schnittstellen für Modelle

Gradio eignet sich, wenn ein Simulations- oder ML-Modell direkt interaktiv ausgeführt werden soll — ohne Datei-Upload-Logik.

```python
# KI-generiert — Gradio Abfluss-Rechner
import gradio as gr

def berechne_abfluss(niederschlag, bodentyp):
    faktor = 0.3 if bodentyp == "Ton" else 0.1
    return f"Abfluss: {niederschlag * faktor:.2f} mm"

demo = gr.Interface(
    fn=berechne_abfluss,
    inputs=[gr.Number(label="Niederschlag [mm]"),
            gr.Dropdown(["Sand", "Ton"],
                        label="Bodentyp")],
    outputs=gr.Text(label="Abfluss [mm]")
)
demo.launch()
```

Streamlit — Daten laden, filtern, visualisieren

→ Sensor-Dashboard, Messwertauswertung

Gradio — Parameter eingeben, Modell ausführen

→ Bemessungstools, Simulations-Schnittstellen

Gradio eignet sich besonders für die Weitergabe von Berechnungsmodellen an Fachkollegen ohne Python-Kenntnisse.

Gradio und Streamlit decken unterschiedliche Anwendungsprofile ab. Streamlit ist ideal für Daten-Dashboards: CSV laden, filtern, visualisieren, exportieren. Gradio ist ideal für Modell-Interfaces: Parameter eingeben, Ergebnis erhalten. Das Modell selbst bleibt dem Nutzer verborgen.

Verifikation gilt auch für Gradio: Stimmt die Rechenformel? Sind Einheiten in Labels klar? Werden ungültige Eingaben (negativer Niederschlag, leeres Feld) korrekt abgefangen?

→ Gradio-Dokumentation: gradio.app

Verifikations-Checkliste

TL;DR

Falsche Daten klar dargestellt ist gefährlicher als richtige Daten unzugänglich. Die Verifikations-Checkliste stellt sicher, dass das KI-generierte Interface die Ingenieuranforderungen erfüllt — bevor es in Betrieb geht.

UI-Verifikations-Checkliste

Diese Checkliste entspricht dem Prinzip aus `08b_CodeReview`: systematische Verifikation statt intuitivem „Sieht gut aus“. Jeder Punkt wird aktiv geprüft — mit echten oder synthetischen Testdaten.

Die vier Kategorien decken ab: **Was steht dran** (Beschriftung), **Was steckt drin** (Datenmapping), **Wie es aussieht** (Darstellung) und **Was passiert im Fehlerfall** (Robustheit).

Ausblick: Block 6 — Persistente Datenhaltung

Das Dashboard funktioniert. Aber:

- 10 Millionen Messwerte — aus Dateien laden?
- 50 Nutzer gleichzeitig — auf eine CSV?
- Historische Abfragen über Monate — durch alle Ordner?

Dateien skalieren nicht.

⟶ Block 6: **Datenbanken** als persistente, skalierbare Datenschicht hinter dem Dashboard.

⟶ `12a_Datenhaltung`: Warum Datenbanken statt Dateien?

Heute: CSV-Datei ⟶ Streamlit

Block 6: Datenbank ⟶ Streamlit

Ein Streamlit-Dashboard ist so gut wie seine Datenquelle.

Für Produktivsysteme: Datenbanken statt CSV-Dateien.

Das Sensor-Dashboard aus dieser Einheit funktioniert hervorragend für einige tausend Datenpunkte aus einer CSV-Datei. In der Ingenieurpraxis sieht die Situation oft anders aus: Sensornetze liefern Millionen von Messwerten, mehrere Benutzer greifen gleichzeitig zu, historische Analysen erfordern Abfragen über lange Zeiträume.

Diese Anforderungen motivieren **Datenbanken** als persistente Datenschicht hinter dem Dashboard. Block 6 (Sitzungen 12–13) führt relationale Datenbanken und SQL ein — die gleiche Streamlit-App kann danach mit einer Datenbankabfrage statt einem CSV-Upload betrieben werden.

Cross-Referenz: `12a_Datenhaltung` motiviert den Übergang von dateibasierter zu datenbankgestützter Datenhaltung im Detail.

Referenzen

Bibliographie